

AULA 6

Laboratório: Persistência de Dados



OBJETIVOS DA AULA



AO FINAL DESSA AULA, VOCÊ VAI SER CAPAZ DE...

- Conectar o Prisma Client dentro do NestJS através de um PrismaService
- Trocar o array em memória do UsersService por consultas reais ao banco
- Implementar create, findMany e findUnique
- Implementar update e delete usando o Prisma Client



CONECTANDO O PRISMA AO NESTJS



PRISMA SERVICE + DRIVER ADAPTER

```
@Injectable()
export class PrismaService
  extends PrismaClient
  implements OnModuleInit, OnModuleDestroy {

  constructor() {
    const adapter = new PrismaBetterSqlite3({
      url: process.env.DATABASE_URL,
    });
    super({ adapter });
  }

  async onModuleInit() { await this.$connect(); }
  async onModuleDestroy() { await this.$disconnect(); }
}
```

POR QUE UM SERVICE PRÓPRIO

- Encapsula o PrismaClient como um provider injetável, gerido pelo container de DI do Nest
- onModuleInit conecta ao banco quando a aplicação sobe; onModuleDestroy desconecta ao encerrar
- O driver adapter (@prisma/adapter-better-sqlite3) é obrigatório na versão atual do Prisma, mesmo pro SQLite
- Registrado num PrismaModule @Global(), disponível pra qualquer service do projeto



OPERAÇÕES DE PERSISTÊNCIA



CREATE, UPDATE, DELETE

```
create(data) {  
  return this.prisma.user.create({ data });  
}  
  
update(id, data) {  
  return this.prisma.user.update({  
    where: { id },  
    data,  
  });  
}  
  
remove(id) {  
  return this.prisma.user.delete({  
    where: { id },  
  });  
}
```

O QUE MUDA EM RELAÇÃO AO ARRAY

- Mesma assinatura de método que já existia desde a Aula 2/4 — só o corpo mudou
- O Controller não muda nem uma linha: continua chamando `userService.create()`, `.update()`, `.remove()`
- Os métodos agora retornam `Promise` — o Nest resolve isso automaticamente
- `update` e `delete` lançam erro do próprio Prisma se o `id` não existir — tratamento na próxima seção



CONSULTAS AO BANCO



FINDMANY E FINDUNIQUE

```
findAll() {  
  return this.prisma.user.findMany();  
}  
  
async findOne(id: number) {  
  const user = await this.prisma.user  
    .findUnique({ where: { id } });  
  
  if (!user) {  
    throw new NotFoundException(  
      `Usuário ${id} não encontrado`  
    );  
  }  
  return user;  
}
```

DIFERENÇA IMPORTANTE

- `findMany()` lista tudo — equivalente ao antigo `this.users`
- `findUnique()` busca um só, por uma chave única (aqui, `id`) — e retorna `null` se não achar
- `null` não é um erro HTTP por si só — por isso o `if(!user)` continua necessário, igual na Aula 4
- A regra de tratamento de erro da Aula 4 não mudou — só a fonte do dado mudou, de array pra banco

